couplr: Optimal Pairing and Matching via Linear Assignment

by Gilles Colling

Abstract Matching two groups of units so that the total cost of the chosen pairs is as small as possible is the linear assignment problem, and it sits underneath causal-inference matching, experimental design, task allocation, sample pairing and image alignment. R users currently meet this problem through two separate families of packages: causal-inference packages that wrap a single fixed solver behind a treated/control interface, and solver packages that expose one algorithm each without preprocessing, constraints or balance diagnostics. The couplr package covers both. A data-frame interface builds distances, applies calipers, blocks and forbidden pairs, and returns analysis-ready matched output together with balance diagnostics and Rosenbaum sensitivity bounds. A matrix interface exposes the solvers directly, along with dual variables that certify optimality, k-best assignments and bottleneck assignment. Nineteen assignment algorithms are implemented from scratch in C++, spanning augmenting-path, auction, cost-scaling and network-flow families, and a dispatcher chooses among them from problem shape, sparsity, cost type and size. Solvers are templated on their cost source, so the same solver body reads a dense matrix or evaluates distances on demand from the covariates; the second path matches fifty thousand units without ever building the distance matrix. On a one-to-one optimal Mahalanobis task, couplr reproduces the balance that MatchIt and optmatch achieve and reaches problem sizes at which both alternatives stop. couplr is available on CRAN.

1 Introduction

Many data-analysis tasks come down to the same question: given two sets of objects and a cost attached to every possible pair, which pairing has the smallest total cost? A researcher building an observational study pairs treated units with controls that resemble them on measured covariates. A scheduler assigns staff to shifts. An ecologist pairs surveyed plots across two time points. An image-processing pipeline maps the pixels of one picture onto the pixels of another. In each case the mathematical object is the linear assignment problem, and the quality of the answer depends on solving it to optimality rather than greedily.

R has good software for both ends of this problem, in two groups that do not meet. On the statistical side, **MatchIt** (Ho et al., 2011), **optmatch** (Hansen, 2007), **Matching** (Sekhon, 2011) and **designmatch** (Zubizarreta et al., 2024) provide preprocessing, matching designs and estimation support for causal inference, with **cobalt** (Greifer, 2025) supplying balance diagnostics across packages. Each routes through a single fixed back end: **MatchIt** delegates optimal matching to **optmatch**, **optmatch** solves a minimum-cost flow problem (Hansen and Klopfer, 2006), **Matching** uses genetic search, and **designmatch** formulates an integer program that in practice wants a commercial solver. On the algorithmic side, **clue** (Hornik, 2024) exposes the Hungarian method through `solve_LSAP()`, and **lpSolve** (Berkelaar et al., 2024) solves the assignment problem as a general linear program. These give the user an optimum for a cost matrix they have already built, and nothing else: no covariate handling, no calipers, no balance table.

A workflow that needs both halves therefore threads several packages by hand, and any choice away from the defaults moves the user into a different corner of that toolchain. Matching on observed covariates rather than on a propensity score, imposing calipers on more than one variable at a time, blocking, or choosing the solver deliberately are all reachable, and each is reachable somewhere else.

The **couplr** package (Colling, 2026) treats the assignment problem itself as the organising object and builds the workflow around it. A user who starts from a data frame with covariates and a treatment indicator, and a user who starts from a cost matrix, enter through different functions and reach the same solver core. The package contributes three things to R:

1. A matching interface in which covariate distance is the primary entry point, with calipers, blocking, forbidden pairs, variable ratios, replacement, balance diagnostics and Rosenbaum sensitivity bounds in one object model.
2. Nineteen assignment algorithms implemented from scratch in C++ and reachable by name, covering augmenting-path, auction, cost-scaling and network-flow families, together with dual variables, k-best assignments, bottleneck assignment and entropy-regularised transport.
3. A dispatcher and a memory model that pick the solver and the cost representation from the problem, so the interface does not change as problems grow from hundreds to tens of thousands of units.

The rest of this article states the problem couplr solves and the algorithm families that solve it, walks through a matching workflow and the designs around it, describes the solver layer, then covers the software design in detail: object model, C++ organisation, dispatch rules, memory modes, verification and dependency footprint. A performance section reports per-solver timings, a balance comparison against **MatchIt** and **optmatch** on the LaLonde data, and a scaling comparison. Results were produced with couplr version 1.5.4.

2 The assignment problem behind matching

Let $C \in \mathbb{R}^{n \times m}$ be a cost matrix whose entry C_{ij} is the cost of pairing row $i \in \{1, \dots, n\}$ with column $j \in \{1, \dots, m\}$. Write $X \in \{0,1\}^{n \times m}$ for the assignment matrix, with $X_{ij} = 1$ when row i is matched to column j. The linear assignment problem is

$$\min_X \sum_{i=1}^{n} \sum_{j=1}^{m} C_{ij} X_{ij} \quad \text{subject to} \quad \sum_j X_{ij} \le 1 \ \forall i, \qquad \sum_i X_{ij} \le 1 \ \forall j. \tag{1}$$

In a matching application, rows are treated units, columns are controls, and C_{ij} is a distance between covariate vectors. In a scheduling application, rows are workers, columns are shifts, and C_{ij} is a cost or the negative of a preference score. Forbidden pairs are encoded as $C_{ij} = \infty$.

The constraint matrix of (1) is totally unimodular, so the linear programming relaxation has integral optima (Birkhoff, 1946; Burkard and Çela, 1999) and the combinatorial problem can be solved by primal-dual methods rather than by branch and bound. The dual assigns a potential u_i to each row and v_j to each column subject to

$$u_i + v_j \le C_{ij} \quad \forall (i,j), \qquad u_i + v_j = C_{ij} \quad \text{whenever } X_{ij} = 1, \tag{2}$$

and a feasible pair (u, v) satisfying (2) certifies that X is optimal. couplr returns these potentials from `assignment_duals()`, so a user can check optimality without trusting the solver, and can read the reduced costs $C_{ij} - u_i - v_j$ as the penalty for forcing any particular pair.

Why more than one algorithm

Every solver of (1) returns the same optimal value, so the choice among them is a performance question rather than a statistical one, and the performance depends on the shape of the problem. couplr groups its solvers into four families and a set of special-purpose routines.

Augmenting-path methods grow a matching one row at a time, each time finding a shortest augmenting path in the residual graph under reduced costs. The Hungarian method (Kuhn, 1955; Munkres, 1957) is the classical member; the Jonker-Volgenant algorithm (Jonker and Volgenant, 1987) adds column reduction and auction-style initialisation, which is what makes it fast in practice on dense problems despite an $O(n^3)$ worst case. `lapmod` is the sparse variant, working from an adjacency list so that forbidden pairs cost nothing to carry.

Auction methods (Bertsekas, 1988) let rows bid for columns, raising prices until no row wants to switch. They converge to optimality once the bid increment ε falls below $1/n$ for integer costs, and ε-scaling gives the useful practical variants. Auction is naturally parallel and tolerant of near-degenerate cost structures where augmenting-path methods spend time on long paths.

Cost-scaling methods solve a sequence of increasingly precise problems, each time halving the scale on which costs are rounded and reusing the previous solution (Goldberg and Kennedy, 1995). The Gabow-Tarjan bit-scaling algorithm (Gabow and Tarjan, 1989) belongs here, and reaches $O(\sqrt{n}\, m \log(n C_{\max}))$, where $C_{\max} = \max_{i,j} |C_{ij}|$. To our knowledge this algorithm did not previously have a publicly available open-source implementation in any language.

Network-flow methods treat (1) as a minimum-cost flow problem on a bipartite network (Ahuja et al., 1993) and apply push-relabel (Goldberg and Tarjan, 1988), cycle cancelling, network simplex, or Orlin's strongly polynomial algorithm (Orlin, 1993). These are the most general formulations, which makes them the right base for variable-ratio designs where a group absorbs several units, and the slowest choice for a plain one-to-one problem.

Special-purpose routines cover cases where the general machinery is wasted: exhaustive enumeration below nine units, Hopcroft-Karp style bipartite matching (Hopcroft and Karp, 1973) when costs are binary or constant, Ramshaw-Tarjan (Ramshaw and Tarjan, 2012) for strongly rectangular problems where padding to a square matrix would dominate the work, and bottleneck assignment, which minimises the largest pair cost instead of the sum.

Two extensions round out the set. Murty's ranking procedure (Murty, 1968) with Lawler's partition scheme (Lawler, 1972) enumerates the k cheapest assignments, which lets a user see how much of the total cost is forced and how much is arbitrary. Sinkhorn iteration (Cuturi, 2013) solves the entropy-regularised relaxation, returning a doubly stochastic coupling rather than a permutation.

3 A matching workflow

The package ships `hospital_staff`, a synthetic personnel dataset that supports a treated/control workflow without external data. The `nurses_extended` table holds 200 units and `controls_extended` holds 300, with age, experience, hourly rate, department, certification level and a full-time indicator.

```
library(couplr)
data(hospital_staff)

treated <- transform(hospital_staff$nurses_extended,   id = nurse_id)
control <- transform(hospital_staff$controls_extended, id = nurse_id)
```

`match_couples()` is the front door. It takes the two data frames, the covariates to match on, and returns the optimal one-to-one pairing under the requested distance. Setting `auto_scale = TRUE` runs the preprocessing step: covariates are screened for degenerate variance and excessive missingness, and a scaling method is chosen so that variables measured in years and variables measured in ordinal levels contribute comparably.

```
covars <- c("age", "experience_years", "certification_level")

m <- match_couples(
  left  = treated,
  right = control,
  vars  = covars,
  auto_scale = TRUE
)
m
```

```
#> Matching Result
#> ===============
#>
#> Method: lap
#> Pairs matched: 200
#> Unmatched (left): 0
#> Unmatched (right): 100
#> Total distance: 59.9496
#>
#> Matched pairs:
#> # A tibble: 200 x 6
#>    left_id right_id distance .age_diff .experience_years_diff
#>    <chr>   <chr>       <dbl>     <dbl>                  <dbl>
#>  1 1       260         0.217        -1                      1
#>  2 2       294         0.175         2                      0
#>  3 3       212         0             0                      0
#>  4 4       444         0.591        -5                      2
#>  5 5       304         0.481        -5                     -1
#>  6 6       273         0             0                      0
#>  7 7       496         0.199         0                      1
#>  8 8       292         0.262         3                      0
#>  9 9       242         0.622        -2                      3
#> 10 10      447         0             0                      0
#> # i 190 more rows
#> # i 1 more variable: .certification_level_diff <int>
```

The result is an S3 object of class `matching_result` with three components. `m$pairs` is a tibble of matched pairs carrying the identifiers, the distance for each pair and a signed per-variable difference column for every matching variable, which makes it possible to see which covariate drives a poor pair

without recomputing anything. `m$unmatched` holds the identifiers left over on each side, and `m$info` records the method used, the number of pairs and the total distance.

Balance is the standard for judging whether a matched sample is usable. `balance_diagnostics()` computes standardised mean differences, variance ratios and Kolmogorov-Smirnov statistics on the matched sample, and `balance_table()` formats them.

```
bal <- balance_diagnostics(m, treated, control, vars = covars)
balance_table(bal)
```

```
#> # A tibble: 3 x 7
#>   Variable `Mean Left` `Mean Right` `Mean Diff` `Std Diff` `Var Ratio` `KS Stat`
#>   <chr>          <dbl>        <dbl>       <dbl>      <dbl>       <dbl>     <dbl>
#> 1 age            38.6         39.8       -1.22      -0.116       1.04      0.075
#> 2 experie~        8.46         8.10       0.355      0.074       1.05      0.06
#> 3 certifi~        1.88         1.9       -0.015     -0.021       0.964     0.015
```

All three covariates land well inside the conventional threshold of 0.1 on the absolute standardised difference (Stuart, 2010). `join_matched()` then returns the analysis-ready frame, one row per pair, with every column of both input tables carried through under `_left` and `_right` suffixes, so an outcome model can be fitted directly on the matched data.

```
matched <- join_matched(m, treated, control)
dim(matched)
```

```
#> [1] 200  21
```

A matched analysis rests on assumptions that no matching method can verify: conditional ignorability, positivity and SUTVA. What matching can offer is a statement about how strong an unmeasured confounder would need to be before the conclusion changes. `sensitivity_analysis()` reports the Rosenbaum bound, the critical odds ratio Γ at which the inference stops being significant (Rosenbaum, 2002). The matched output is also designed to feed an outcome regression, so that the estimate is doubly robust rather than resting on the match alone (Stuart, 2010).

4 Constraints, designs and interoperability

Reshaping the cost matrix

Three mechanisms restrict which pairs are admissible, and all three work by writing non-finite entries into the cost matrix before it reaches the solver. A global `max_distance` forbids any pair beyond a distance ceiling. Per-variable `calipers` forbid pairs whose raw difference on a named variable exceeds a threshold, independently of the distance metric in use. Explicit forbidden pairs are passed as non-finite entries by the user. Every solver recognises a non-finite entry as a missing edge, and reports infeasibility rather than returning a pairing that quietly violates the constraint.

```
m_cal <- match_couples(
  treated, control, vars = covars,
  auto_scale   = TRUE,
  calipers     = list(age = 3, experience_years = 2),
  max_distance = 1.5
)
c(pairs = nrow(m_cal$pairs), unmatched_left = length(m_cal$unmatched$left))
```

```
#>          pairs unmatched_left
#>            180             20
```

The caliper costs pairs: 180 of the 200 treated units keep a partner. This is the intended behaviour of a caliper and the reason the unmatched identifiers are returned rather than dropped, since the units that fail to match define the population the estimate actually applies to.

Blocking is the other structural constraint. `block_id` restricts matching to run within levels of a factor, and `matchmaker()` constructs blocks when no grouping variable exists, either from an existing variable or by k-means or hierarchical clustering on chosen block variables. The solver is called once per block, and blocked designs report per-block balance so that imbalance inside one stratum is not averaged away across the others.

Matching designs

Beyond one-to-one matching, `match_couples()` takes `ratio` for k-to-one designs and `replace` for matching with replacement. Five further designs have their own entry points, each returning an object in the same family:

- `ps_match()` estimates a propensity score (Rosenbaum and Rubin, 1983) and matches on it, with the caliper expressed in standard deviations of the linear predictor.
- `full_match()` assigns every unit to a matched group of variable size, so no unit is discarded. The optimal path solves a minimum-cost flow problem using Dijkstra's algorithm with Johnson potentials (Johnson, 1977); a greedy two-pass heuristic is available for large problems.
- `cem_match()` implements coarsened exact matching (Iacus et al., 2012), binning covariates and matching exactly on the binned strata.
- `subclass_match()` divides the sample into propensity-score subclasses and returns subclass weights for ATT or ATE estimation.
- `cardinality_match()` targets the cardinality-matching objective (Zubizarreta et al., 2014), the largest matched sample meeting prespecified balance constraints. The implementation is an iterative pruning heuristic rather than the mixed-integer program: it starts from a full match and repeatedly removes the pairs contributing most to the worst-balanced variable until every standardised difference falls under the threshold. It therefore approximates the cardinality-matching solution and does not certify maximality.

Interoperability

couplr does not ask a user to abandon the packages already in their workflow. `as_matchit()` converts a couplr result into a `matchit` object, so downstream code written against **MatchIt** keeps working, and `match_data()` returns data in the layout that **MatchIt** users expect. `bal.tab()` methods are registered for the couplr result classes, so **cobalt** produces its usual balance output on a couplr match. Weighted matched data flows into **marginaleffects** (Arel-Bundock, 2025) for estimation.

5 The solver layer

A user who already has a cost matrix does not need any of the matching machinery. `lap_solve()` and `assignment()` take a matrix and return the optimal assignment; `lap_solve()` also accepts a long data frame of source, target and cost columns, which is the natural shape when costs arrive from a database.

```
set.seed(1)
cost <- matrix(sample.int(100, 25, replace = TRUE), nrow = 5)
res  <- lap_solve(cost)
res
```

```
#> Assignment Result
#> =================
#>
#> # A tibble: 5 x 3
#>   source target  cost
#>    <int>  <int> <int>
#> 1      1      4     7
#> 2      2      2    14
#> 3      3      1     1
#> 4      4      3    54
#> 5      5      5    44
#>
#> Total cost: 120
#> Method: bruteforce
```

The `method` argument names any of the nineteen solvers; `"auto"` is the default. Dual potentials come from `assignment_duals()`, which returns the vectors u and v of (2) alongside the assignment, so optimality can be certified independently:

```
d <- assignment_duals(cost)
all(outer(d$u, d$v, "+") <= cost + 1e-9)
```

```
#> [1] TRUE
```

Three extensions operate on the same matrix. `lap_solve_kbest()` returns the k cheapest assignments in increasing order of cost, which shows how much of a solution is forced by the data. `bottleneck_assignment()` minimises the largest individual pair cost, the right objective when the worst pair rather than the average pair is what matters. `lap_solve_batch()` solves a stack of independent problems, optionally across threads. `sinkhorn()` returns the entropy-regularised coupling, and `sinkhorn_to_assignment()` rounds it to a permutation.

```
kb <- lap_solve_kbest(cost, k = 3)
tapply(kb$total_cost, kb$rank, unique)
```

```
#>   1   2   3
#> 120 120 150
```

The two cheapest assignments both cost 120 and the third costs 150, so the optimum here is attained by two distinct pairings. A matching study reading the same output would learn that its matched sample is not unique, which stays invisible when a solver returns one assignment.

6 Software design

Layers and object model

The package is organised in three layers. The first converts user input into a cost matrix: it validates the data frames, screens covariates, applies scaling and weights, computes distances, and writes constraints into the matrix. The second solves (1). The third builds result objects and the methods that consume them.

The object model is S3 throughout. Matching results carry the class vector `c("matching_result", "couplr_result")`, with the design-specific classes (`full_matching_result`, `cem_result`, `subclass_result`) sharing the `couplr_result` parent. The shared parent is what makes the diagnostic and conversion layer generic: `balance_diagnostics()`, `join_matched()`, `match_data()` and the `bal.tab()` methods dispatch on it once rather than being rewritten per design. S4 or R6 would have bought encapsulation the package does not need, at the cost of making result objects harder to inspect interactively and harder to serialise. Results are plain lists of tibbles, so a user who wants something the package does not provide can reach in and take it.

Solver results are a separate family. `lap_solve()` returns a tibble subclassed as `lap_solve_result` with `source`, `target` and `cost` columns, which means the optimum is immediately usable in **ggplot2** (Wickham, 2016) or a join without an extraction step, while `get_total_cost()` and `get_method_used()` recover the metadata carried in attributes.

C++ organisation

All nineteen solvers are written from scratch in C++ and exposed through **Rcpp** (Eddelbuettel and François, 2011). The `src` tree separates concerns: `src/core` holds the cost-source types and shared utilities with no Rcpp dependency, `src/solvers` holds one algorithm per file with a thin `_rcpp.cpp` wrapper each, and `src/interface` holds the conversion between R objects and the internal types. The separation means the solver bodies are ordinary C++ that can be compiled and tested outside R.

The design decision that carries the most weight is that solvers are templated on their cost source rather than written against a concrete matrix. A cost source has to provide `at(i, j)`, `allowed(i, j)` and `empty()`. Two types satisfy this interface: `CostMatrix`, which owns a dense flat array plus a mask, and `LazyCostMatrix`, which owns the two feature matrices and computes C_{ij} on demand from the requested metric, applying calipers and the distance ceiling in `allowed()`. A handful of hot loops index the dense mask array directly for speed, which the lazy type cannot support; these are guarded by a `supports_raw_mask` trait and an `if constexpr` branch rather than by duplicating the solver. One solver body therefore serves both cost representations, and there is no second implementation to keep in step.

`LazyCostMatrix` also bakes in the transformations that the dense path applies in a separate preparation pass, namely mapping forbidden entries to a large sentinel and negating costs when maximising. Doing this at construction is what allows the untouched solver body to work on either type.

Automatic solver selection

`method = "auto"` inspects the problem and picks a solver. The rules are deliberately few and readable, and derive from the benchmark in the next section:

1. Problems with at most eight rows and eight columns go to exhaustive enumeration, which is exact and faster than setting up a general solver.
2. Cost matrices whose finite entries are constant or lie in $\{0, 1\}$ go to the Hopcroft-Karp style routine, which exploits the absence of a real cost scale.
3. Matrices with more than half of their entries non-finite go to `lapmod`, the sparse Jonker-Volgenant variant, which carries forbidden edges in its adjacency structure at every size.
4. Strongly rectangular problems, with at least three times as many columns as rows, go to shortest augmenting paths, avoiding the padding a square-oriented solver would need.
5. Everything else goes to Jonker-Volgenant.

Rectangular problems are transposed internally so that the solver always sees at least as many columns as rows, and the assignment is mapped back afterwards; the user never sees the transpose.

Rules 2 and 3 read the entries, and so does the rejection of `NaN` inputs that runs for every method, so choosing a solver means a pass over the cost matrix. Expressing that pass in R cost an allocation the size of the matrix for each test, through `any(is.nan())`, `range(finite = TRUE)` and `mean(is.na() | is.infinite())`, and the overhead made `"auto"` up to twice as slow as naming the solver it would pick. A single C++ pass now returns every quantity the rules need, with no allocation and no second scan: the binary-cost test is a branch inside the same loop rather than a `unique()` over the finite entries, and an integer cost matrix is read as integers rather than coerced. That leaves the inspection a linear read in front of a superlinear solve, so its share of the runtime falls as problems grow, and the dashed line in Figure 1 sits on the solver the dispatcher selects. The pass is skippable for callers who already know the shape of their problem, by naming the method.

Memory modes

The dense representation of the cost matrix is the binding constraint on problem size long before runtime is. A 100,000 by 100,000 problem needs 80 GB as doubles; the same units described by 100 covariates need 80 MB. couplr exposes this as `memory_mode`, with values `"dense"`, `"lazy"` and `"auto"`.

Under `"lazy"`, no cost matrix is built. The solver holds the feature matrices and evaluates distances as it requests them, which trades arithmetic for memory and returns the same assignment the dense path returns. The lazy path covers Jonker-Volgenant and auction with the built-in metrics, together with calipers and the distance ceiling.

Under `"auto"`, the package decides. Below ten million cells, about 80 MB dense, it skips the check entirely, since probing free memory on every ordinary problem would be pure overhead. Above that, it estimates the dense footprint with a factor of four over the raw eight bytes per cell, covering the copies made during conversion and preparation that can coexist under garbage-collection lag, and compares that against available system memory. Available memory is read per platform, from `MemAvailable` on Linux, from `vm_stat` page counts on macOS, including inactive and speculative pages because the kernel keeps almost nothing on the free list, and from `Win32_OperatingSystem` on Windows. If the estimate exceeds half of what is available, `"auto"` switches to the lazy path where one exists and warns explicitly where one does not, naming blocking, greedy matching or a smaller problem as the alternatives. Detection failure is treated as unknown rather than as success: a fixed 4 GB threshold takes over, so the guard never silently disappears on an unrecognised platform.

Verification

Statistical smoke tests establish that a function returns an object of the right shape. For a solver, the corresponding standard is whether the returned assignment is the optimal one. Every optimal solver in couplr is checked against exhaustive enumeration on randomised instances covering integer and fractional costs, square and rectangular shapes, minimisation and maximisation, and matrices containing forbidden edges. The brute-force reference is the same `bruteforce` solver the dispatcher uses below nine units, so the verified path and the shipped path are one implementation. The package carries 125 test files in total, covering the R layer, the C++ wrappers, dispatch rules and the matching designs.

Dual feasibility gives a second, independent check that a user can run on their own problem, since (2) can be verified in $O(nm)$ without solving anything, as shown in the solver section above.

Dependency footprint

couplr declares **Rcpp**, **tibble** (Müller and Wickham, 2025), **dplyr**, **rlang**, **purrr** and `methods` in `Imports`, and `Rcpp` alone in `LinkingTo`. The recursive closure of hard dependencies is 17 non-base packages, so a default installation brings 18 packages including couplr itself. The comparable figures on the same day were 18 for **optmatch**, 8 for **MatchIt** and 7 for **designmatch**.

Two choices sit behind that number. First, no external solver is used: all nineteen algorithms are package source, so there is no dependency on an LP or MIP library, no system solver to install, and no licence question of the kind that arises when an integer-programming design reaches for a commercial back end. Second, everything that serves a single feature lives in `Suggests` and is loaded at call time. The animation and image-morphing functions, the plotting helpers, the interoperability layer for **MatchIt**, **optmatch** and **cobalt**, and the parallel back end are all optional in this sense, so a user who only wants to match pays for none of them. `LinkingTo` deserves particular care, since every entry there forces a compile of that package before couplr can be built from source; couplr declares only what its headers actually include.

7 Performance

Solver benchmark

Figure 1 reports median wall-clock solve time against problem size for all nineteen solvers, grouped by family. Costs are integers drawn uniformly from $[1, 10{,}000]$ on square matrices, timings are medians of five replicates on a single core of an Apple M4 Pro, and the slower families are capped at smaller sizes so that the scalable solvers can be measured to $n = 5000$ within a reasonable budget.

The picture is consistent across families. Jonker-Volgenant with its warm start is the fastest general-purpose solver at every measured size, which is why it is the dispatcher's default. The auction and cost-scaling families are competitive at small sizes and fall behind on dense uniform costs, where their advantage, tolerance of degenerate structure and a better asymptotic bound in the Gabow-Tarjan case, does not pay. General flow formulations are the slowest, as expected for machinery that solves a larger problem than the one posed. The two special-purpose routines are timed on the inputs they exist for and win decisively there: exhaustive enumeration below nine units, and the Hopcroft-Karp style solver on binary costs, where at $n = 5000$ it runs about twice as fast as Jonker-Volgenant does on uniform integer costs of the same size.

These orderings hold for dense uniform integer costs on square matrices. They are the basis of the dispatch rules, and they are also the reason the dispatcher has rules at all rather than one default: sparsity, rectangularity and degenerate cost scales each move the answer.

Balance against established alternatives

Comparing packages on speed alone would be beside the point if they did not agree on the answer. Figure 2 fixes the task, one-to-one optimal matching on a Mahalanobis distance with pooled within-group covariance, and runs it on the LaLonde NSW data (LaLonde, 1986; Dehejia and Wahba, 1999), 185 treated units, 429 controls and eight covariates. The pooled within-group convention is the one `optmatch::match_on()` uses by default and the one couplr adopts.

The three packages produce identical absolute standardised mean differences to three decimal places, so the matched points coincide and one is plotted. Five of the eight covariates fall below 0.1 after matching; `married` and 1974 earnings sit just above at 0.124 and 0.128, and the indicator for Black respondents drops from 1.757 to 1.053 without approaching the threshold, which is a property of this dataset rather than of any package: the control pool does not contain enough Black respondents for a one-to-one match to balance that indicator.

Scaling

Table 1 reports median wall-clock time on synthetic problems with the same eight-covariate structure, at six sizes with a one-to-two ratio of treated to control units. Every package materialises the distance matrix here, so couplr is timed with `memory_mode = "dense"`.

The margin widens with problem size. couplr is roughly nine times faster than **optmatch** at $n = 500$ and twenty-four times faster at $n = 20{,}000$, where it finishes in 11.4 seconds against 4.7 minutes. **MatchIt** runs between eleven and twenty-four times slower up to $n = 10{,}000$, which is the largest size it completes before `matchit(method = "optimal")` aborts inside the **optmatch** back end

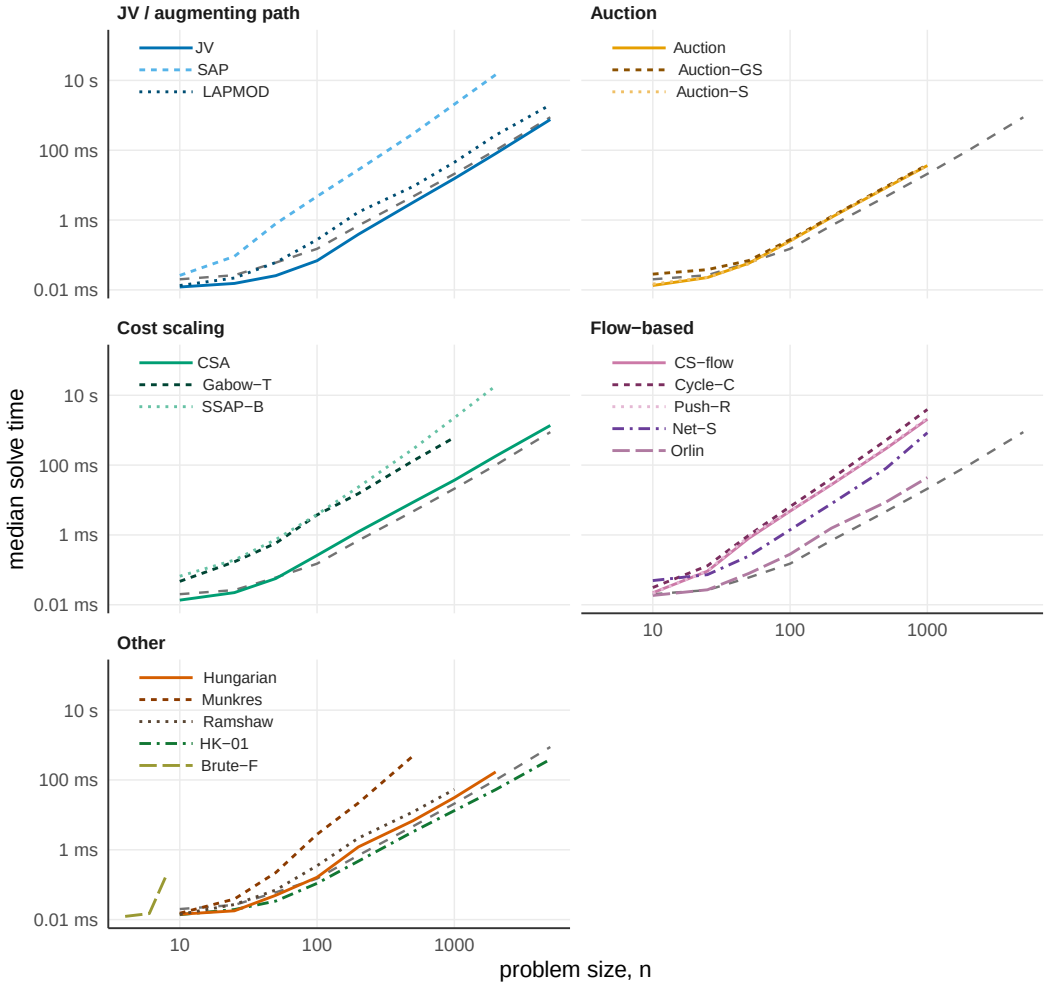

Figure 1: Median wall-clock solve time against problem size for the nineteen assignment solvers in couplr, grouped by algorithm family on shared log-log axes. Seventeen solvers are timed on square integer cost matrices with entries drawn uniformly from 1 to 10,000. The two special-purpose solvers in the Other panel run on their own inputs and are not comparable to the rest: HK-01 is timed on binary cost matrices, and Brute-F only up to n = 8. The dashed grey line repeated in every panel is the automatic dispatcher on the uniform integer costs. Medians of five replicates on a single core of an Apple M4 Pro. Cubic-time and general flow solvers are capped at smaller sizes, which is why their lines end early.

Table 1: One-to-one optimal Mahalanobis matching, median wall-clock time by problem size. Treated to control ratio 1:2, eight covariates, pooled within-group covariance, single core with single-threaded BLAS on an Apple M4 Pro. Medians of 5, 5, 3, 3, 1 and 1 replicates for the six rows. The optmatch option max.problem.size was set to Inf for the two largest sizes. int overflow marks an integer-size overflow inside the optmatch back end reached through MatchIt; timeout marks a replicate exceeding the 300-second cap.

Problem size	couplr	optmatch	MatchIt
167 + 333	11 ms	99 ms	117 ms
667 + 1,333	144 ms	1.7 s	2.12 s
1,667 + 3,333	742 ms	12.9 s	15.7 s
3,333 + 6,667	3.08 s	59.3 s	73.4 s
6,667 + 13,333	11.4 s	280 s	int overflow
16,667 + 33,333	78.5 s	timeout	timeout

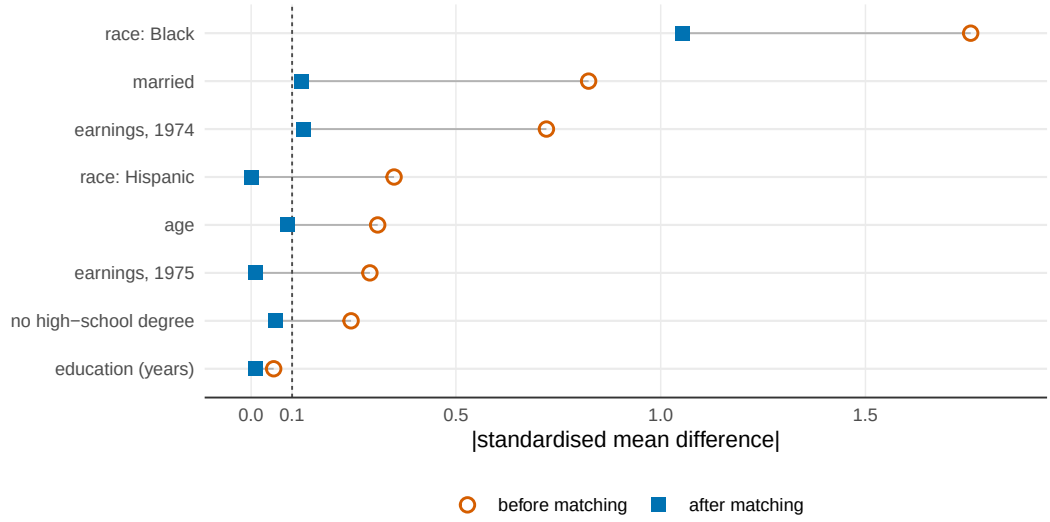

Figure 2: Absolute standardised mean differences on the eight LaLonde NSW covariates, before and after one-to-one optimal Mahalanobis matching with pooled within-group covariance. couplr, MatchIt and optmatch agree to three decimal places after matching, so a single matched point is shown per covariate. The dashed line marks the conventional 0.1 threshold. The indicator for Black respondents starts far outside the plotted range at 1.757 and remains at 1.053 after matching, a residual imbalance that no one-to-one matcher can resolve on this data.

Table 2: Differentiating feature coverage. partial marks a feature that is achievable but not through a first-class user-facing interface: optmatch calipers threshold one distance object at a time, so per-variable calipers require combining objects, and both alternatives handle unequal group sizes through discarding or variable-ratio designs rather than natively.

Feature	couplr	MatchIt	optmatch
Per-variable calipers from a named vector	yes	yes	partial
Rectangular problems without padding	yes	partial	partial
Sparse cost support	yes	no	yes
k-best assignments	yes	no	no
Bottleneck (minimax) assignment	yes	no	no
Rosenbaum sensitivity bounds	yes	no	no
User-selectable assignment algorithm	19 solvers	fixed	fixed

with an integer-size overflow. At $n = 50{,}000$ couplr finishes in 79 seconds and both alternatives exceed the 300-second per-replicate cap.

The reason is dispatch rather than a faster inner loop in any single algorithm. **optmatch** solves a minimum-cost flow problem, which is the right formulation for the full and variable-ratio designs it is built around and more machinery than a plain one-to-one problem needs; couplr's dispatcher sends this dense square problem to Jonker-Volgenant.

The lazy path removes the memory ceiling as well. The same $n = 50{,}000$ match under `memory_mode = "lazy"` completes in 61 seconds without ever building the distance matrix, returning the assignment the dense path returns. At $n = 20{,}000$ it takes 6.3 seconds against 11.4 for the dense path, so on these problems recomputing distances is cheaper than materialising and traversing the matrix.

Feature coverage

Table 2 lists the features that separate the three packages. Rows where all three offer the feature, covariate distance, propensity-score matching, exact blocking and a cost-matrix entry point, are omitted.

8 Discussion

couplr is built on the observation that one optimisation problem sits underneath a set of tasks that R currently treats as unrelated. Making the assignment problem the organising object, rather than a hidden step inside a matching function, is what lets a single package serve a causal-inference user starting from covariates and an operations-research user starting from a cost matrix. The design consequences follow from that decision: an object model with a shared parent class so diagnostics are written once, solvers templated on their cost source so the dense and lazy paths cannot drift apart, and a dispatcher that turns the solver zoo into an implementation detail for users who do not want to think about it.

Several limitations are worth stating. The lazy cost path covers Jonker-Volgenant and auction with the built-in metrics; other solvers and user-supplied distance functions still require a dense matrix, and `full_match()` uses a flow back end that has no lazy path yet. `cardinality_match()` is a pruning heuristic and does not certify that the matched sample it returns is the largest one satisfying the balance constraints, which the mixed-integer formulation would. The dispatch rules are calibrated on dense uniform integer costs and on the sparsity and rectangularity boundaries that follow from them; cost structures far from that regime may be better served by naming a solver explicitly, which is why every solver stays reachable by name. Matching on observed covariates cannot address unmeasured confounding, and the sensitivity analysis quantifies exposure to it rather than removing it.

Work in progress extends the lazy representation to more solvers, adds a minimum-cost flow path for variable-ratio designs under the same templated cost interface, and widens the benchmark beyond uniform integer costs so that the dispatch rules can be recalibrated on structured and clustered cost matrices.

couplr is available on CRAN and developed at `https://github.com/gcol33/couplr`, with documentation at `https://gillescolling.com/couplr/`. Bug reports and feature requests go through the GitHub issue tracker.

9 Acknowledgements

No external funding supported this work.

Bibliography

R. K. Ahuja, T. L. Magnanti, and J. B. Orlin. *Network Flows: Theory, Algorithms, and Applications*. Prentice Hall, Englewood Cliffs, NJ, 1993. [p2]

V. Arel-Bundock. *marginaleffects: Predictions, Comparisons, Slopes, Marginal Means, and Hypothesis Tests*, 2025. URL https://CRAN.R-project.org/package=marginaleffects. R package version 0.29.0. [p5]

M. Berkelaar et al. *lpSolve: Interface to Lp_solve v. 5.5 to Solve Linear/Integer Programs*, 2024. URL https://CRAN.R-project.org/package=lpSolve. R package version 5.6.23. [p1]

D. P. Bertsekas. The auction algorithm: A distributed relaxation method for the assignment problem. *Annals of Operations Research*, 14:105–123, 1988. doi: 10.1007/BF02186476. [p2]

G. Birkhoff. Tres observaciones sobre el algebra lineal. *Universidad Nacional de Tucumán Revista Series A*, 5:147–151, 1946. [p2]

R. E. Burkard and E. Çela. Linear assignment problems and extensions. *Handbook of Combinatorial Optimization*, pages 75–149, 1999. doi: 10.1007/978-1-4757-3023-4_2. [p2]

G. Colling. *couplr: Optimal Pairing and Matching via Linear Assignment*, 2026. URL https://CRAN.R-project.org/package=couplr. R package version 1.5.4. [p1]

M. Cuturi. Sinkhorn distances: Lightspeed computation of optimal transport. In *Advances in Neural Information Processing Systems*, volume 26, pages 2292–2300, 2013. URL https://papers.nips.cc/paper_files/paper/2013/hash/af21d0c97db2e27e13572cbf59eb343d-Abstract.html. [p3]

R. H. Dehejia and S. Wahba. Causal effects in nonexperimental studies: Reevaluating the evaluation of training programs. *Journal of the American Statistical Association*, 94(448):1053–1062, 1999. doi: 10.1080/01621459.1999.10473858. [p8]

D. Eddelbuettel and R. François. Rcpp: Seamless R and C++ integration. *Journal of Statistical Software*, 40(8):1–18, 2011. doi: 10.18637/jss.v040.i08. [p6]

H. N. Gabow and R. E. Tarjan. Faster scaling algorithms for network problems. *SIAM Journal on Computing*, 18(5):1013–1036, 1989. doi: 10.1137/0218069. [p2]

A. V. Goldberg and R. Kennedy. An efficient cost scaling algorithm for the assignment problem. *Mathematical Programming*, 71(2):153–177, 1995. doi: 10.1007/BF01585996. [p2]

A. V. Goldberg and R. E. Tarjan. A new approach to the maximum-flow problem. *Journal of the ACM*, 35(4):921–940, 1988. doi: 10.1145/48014.61051. [p2]

N. Greifer. cobalt: Covariate balance tables and plots. *Journal of Open Source Software*, 10(109):7610, 2025. doi: 10.21105/joss.07610. [p1]

B. B. Hansen. Optmatch: Flexible, optimal matching for observational studies. *R News*, 7(2):18–24, 2007. URL https://journal.r-project.org/articles/RN-2007-014/. [p1]

B. B. Hansen and S. O. Klopfer. Optimal full matching and related designs via network flows. *Journal of Computational and Graphical Statistics*, 15(3):609–627, 2006. doi: 10.1198/106186006X137047. [p1]

D. E. Ho, K. Imai, G. King, and E. A. Stuart. MatchIt: Nonparametric preprocessing for parametric causal inference. *Journal of Statistical Software*, 42(8):1–28, 2011. doi: 10.18637/jss.v042.i08. [p1]

J. E. Hopcroft and R. M. Karp. An $n^{5/2}$ algorithm for maximum matchings in bipartite graphs. *SIAM Journal on Computing*, 2(4):225–231, 1973. doi: 10.1137/0202019. [p2]

K. Hornik. *clue: Cluster Ensembles*, 2024. URL https://CRAN.R-project.org/package=clue. R package version 0.3-66. [p1]

S. M. Iacus, G. King, and G. Porro. Causal inference without balance checking: Coarsened exact matching. *Political Analysis*, 20(1):1–24, 2012. doi: 10.1093/pan/mpr013. [p5]

D. B. Johnson. Efficient algorithms for shortest paths in sparse networks. *Journal of the ACM*, 24(1): 1–13, 1977. doi: 10.1145/321992.321993. [p5]

R. Jonker and T. Volgenant. A shortest augmenting path algorithm for dense and sparse linear assignment problems. *Computing*, 38(4):325–340, 1987. doi: 10.1007/BF02278710. [p2]

H. W. Kuhn. The Hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2 (1-2):83–97, 1955. doi: 10.1002/nav.3800020109. [p2]

R. J. LaLonde. Evaluating the econometric evaluations of training programs with experimental data. *The American Economic Review*, 76(4):604–620, 1986. URL https://www.jstor.org/stable/1806062. [p8]

E. L. Lawler. A procedure for computing the K best solutions to discrete optimization problems and its application to the shortest path problem. *Management Science*, 18(7):401–405, 1972. doi: 10.1287/mnsc.18.7.401. [p3]

K. Müller and H. Wickham. *tibble: Simple Data Frames*, 2025. URL https://CRAN.R-project.org/package=tibble. R package version 3.3.0. [p8]

J. Munkres. Algorithms for the assignment and transportation problems. *Journal of the Society for Industrial and Applied Mathematics*, 5(1):32–38, 1957. doi: 10.1137/0105003. [p2]

K. G. Murty. An algorithm for ranking all the assignments in order of increasing cost. *Operations Research*, 16(3):682–687, 1968. doi: 10.1287/opre.16.3.682. [p3]

J. B. Orlin. A faster strongly polynomial minimum cost flow algorithm. *Operations Research*, 41(2): 338–350, 1993. doi: 10.1287/opre.41.2.338. [p2]

L. Ramshaw and R. E. Tarjan. On minimum-cost assignments in unbalanced bipartite graphs. Technical Report HPL-2012-40R1, HP Laboratories, 2012. URL https://www.hpl.hp.com/techreports/2012/HPL-2012-40R1.html. [p2]

P. R. Rosenbaum. *Observational Studies*. Springer Series in Statistics. Springer, 2 edition, 2002. doi: 10.1007/978-1-4757-3692-2. [p4]

P. R. Rosenbaum and D. B. Rubin. The central role of the propensity score in observational studies for causal effects. *Biometrika*, 70(1):41–55, 1983. doi: 10.1093/biomet/70.1.41. [p5]

J. S. Sekhon. Multivariate and propensity score matching software with automated balance optimization: The Matching package for R. *Journal of Statistical Software*, 42(7):1–52, 2011. doi: 10.18637/jss.v042.i07. [p1]

E. A. Stuart. Matching methods for causal inference: A review and a look forward. *Statistical Science*, 25(1):1–21, 2010. doi: 10.1214/09-STS313. [p4]

H. Wickham. *ggplot2: Elegant Graphics for Data Analysis*, 2016. URL https://ggplot2.tidyverse.org. [p6]

J. R. Zubizarreta, R. D. Paredes, and P. R. Rosenbaum. Matching for balance, pairing for heterogeneity in an observational study of the effectiveness of for-profit and not-for-profit high schools in Chile. *The Annals of Applied Statistics*, 8(1):204–231, 2014. doi: 10.1214/13-AOAS713. [p5]

J. R. Zubizarreta, C. Kilcioglu, and J. P. Vielma. *designmatch: Matched Samples that are Balanced and Representative by Design*, 2024. URL https://CRAN.R-project.org/package=designmatch. R package version 0.5.4. [p1]

Gilles Colling
University of Vienna
Department of Botany and Biodiversity Research
Rennweg 14, 1030 Vienna, Austria
https://gillescolling.com
ORCiD: 0000-0003-3070-6066
gilles.colling051@gmail.com